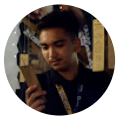
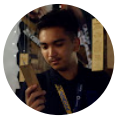
AI Finance Agent with Amazon Bedrock



JUANITO RAMOS II


JUANITO RAMOS II
NextWork Student

Introducing Today's Project!

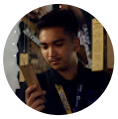
In this project, I will build a personal finance advisor AI agent powered by Amazon Bedrock that uses Code Interpreter to autonomously write and execute Python code, analyzing a CSV of transactions and producing spending breakdowns and charts on demand, to explore and learn how Amazon Bedrock works.

Key services and concepts

I learned how to use Amazon Bedrock Agents to build an AI finance advisor and how Code Interpreter allows an agent to write and execute Python code to analyze transaction data. I also learned about prompt engineering, agent preparation and testing, and AgentCore Memory with session summarization, which allows the agent to remember important information across different sessions.

Challenges and wins

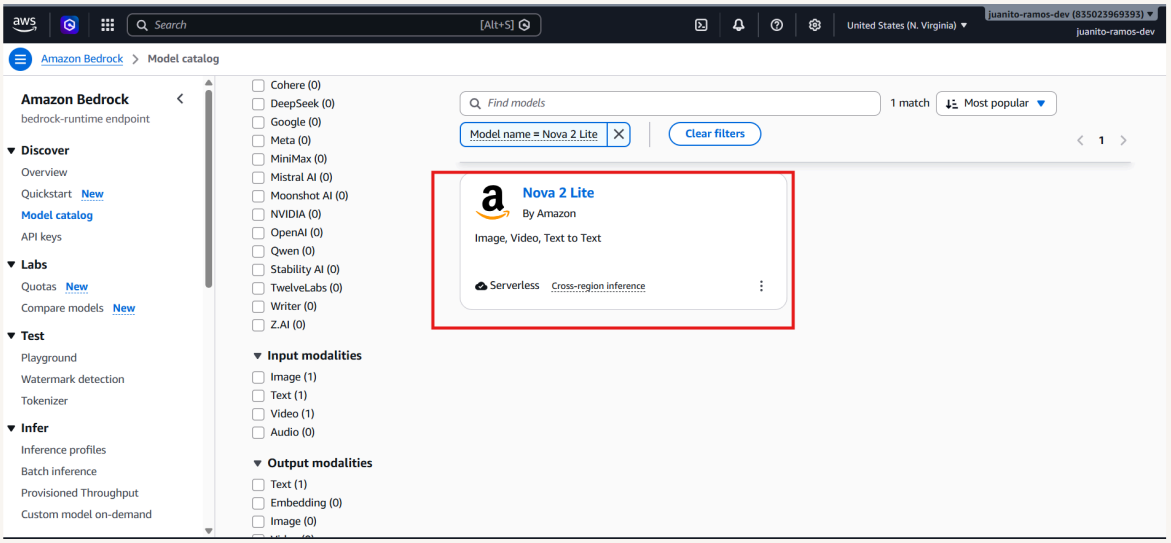
It took me around 1–2 hours to complete the project, including configuring the agent, enabling Code Interpreter, testing the transaction analysis, improving the instructions, and setting up and testing AgentCore Memory.



JUANITO RAMOS II
NextWork Student

Exploring Amazon Bedrock and Foundation Models

In this step, I'm setting up Amazon Bedrock as the foundation for my finance advisor AI agent. I'm exploring the available foundation models and selecting Amazon Nova 2 Lite as the model that will power the agent. I'll also preview the Code Interpreter capability, which will allow the agent to generate and execute Python code for analyzing transaction data and creating visualizations. This step helps me understand how Amazon Bedrock provides managed access to generative AI models and how its capabilities can be used to build practical AI applications without managing the underlying AI infrastructure.



Understanding foundation models

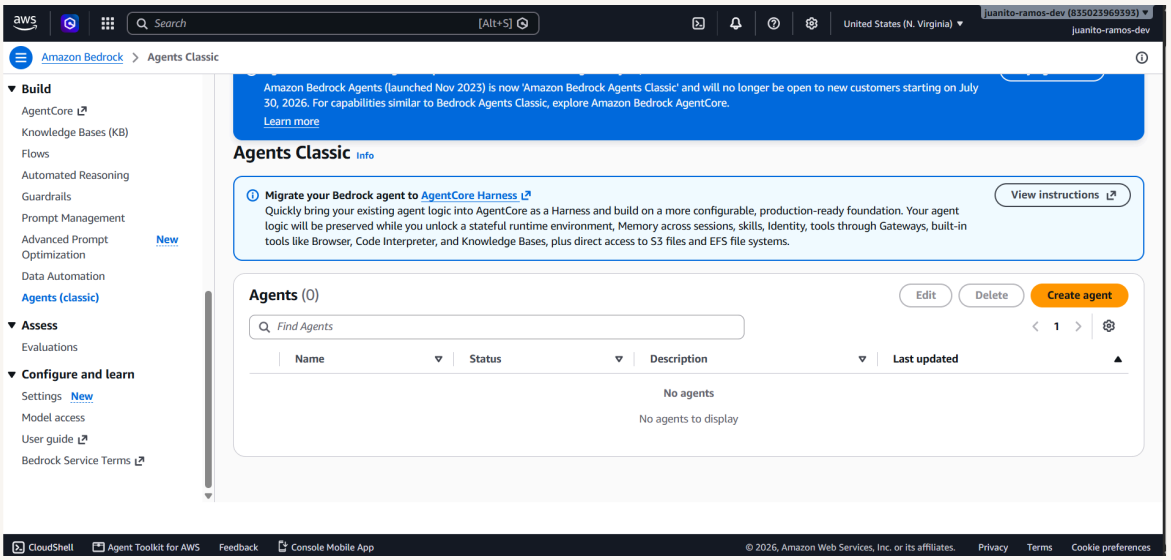
A foundation model is a general-purpose AI model that has been trained on a large amount of data and can be adapted to perform different tasks, such as answering questions, summarizing information, analyzing data, and generating

It gives the agent the language understanding needed to interpret user requests and work with the Code Interpreter to perform data analysis. Using a managed model through Amazon Bedrock also allows me to focus on building the application and agent workflow without having to manage or deploy the underlying AI model infrastructure myself.

Discovering Bedrock Agents

A Bedrock Agent is an AI application that can reason through and complete multi-step tasks autonomously. Unlike a simple chatbot that mainly responds to prompts with generated text, an agent can decide what actions are needed, use tools such as Code Interpreter, and evaluate the results to complete a task. In this project, the Bedrock Agent can understand my request, determine that it needs to analyze the transaction data, generate and execute Python code, and return the results. This makes it more than just a chatbot because it can take actions and use tools to accomplish a specific goal.

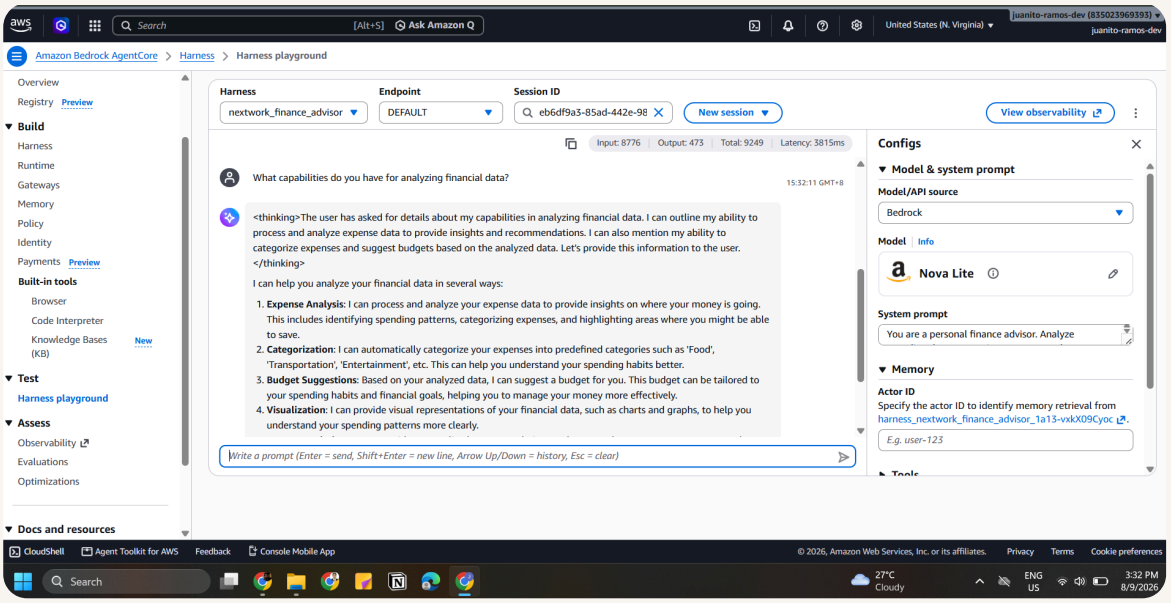
JUANITO RAMOS II
NextWork Student



JUANITO RAMOS II
NextWork Student

Creating the AI Finance Agent

In this step, I'm building the core of my personal finance AI agent using Amazon Bedrock. The original project instructions use Bedrock Agents Classic, but AWS has since placed Agents Classic into maintenance mode and stopped allowing new customers to create agents. Instead, AWS now recommends Amazon Bedrock AgentCore as the newer foundation for building and deploying AI agents. Because of this change, I'm adapting the project to the current AWS architecture while keeping the same goal: building a finance advisor that can understand user requests, analyze transaction data, and use Code Interpreter to write and execute Python code. This gives me hands-on experience not only with building AI agents, but also with navigating changes in cloud services and adapting an existing architecture to newer AWS technologies.

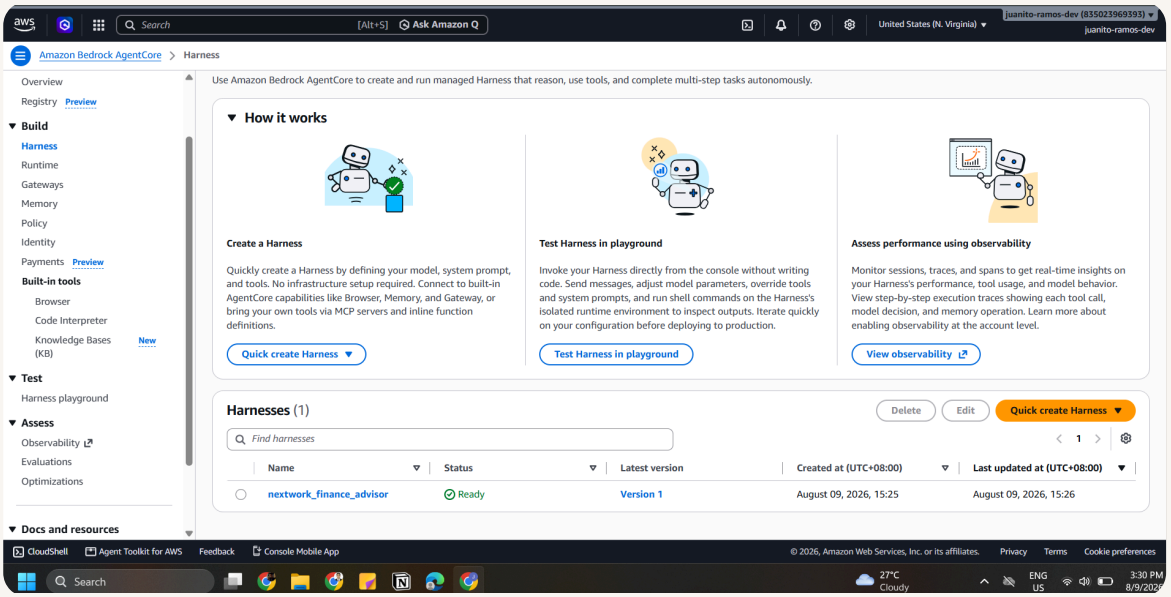


Crafting the agent instructions

I instructed my agent to act as a personal finance advisor that analyzes spending data, categorizes expenses, and suggests budgets. I later improved

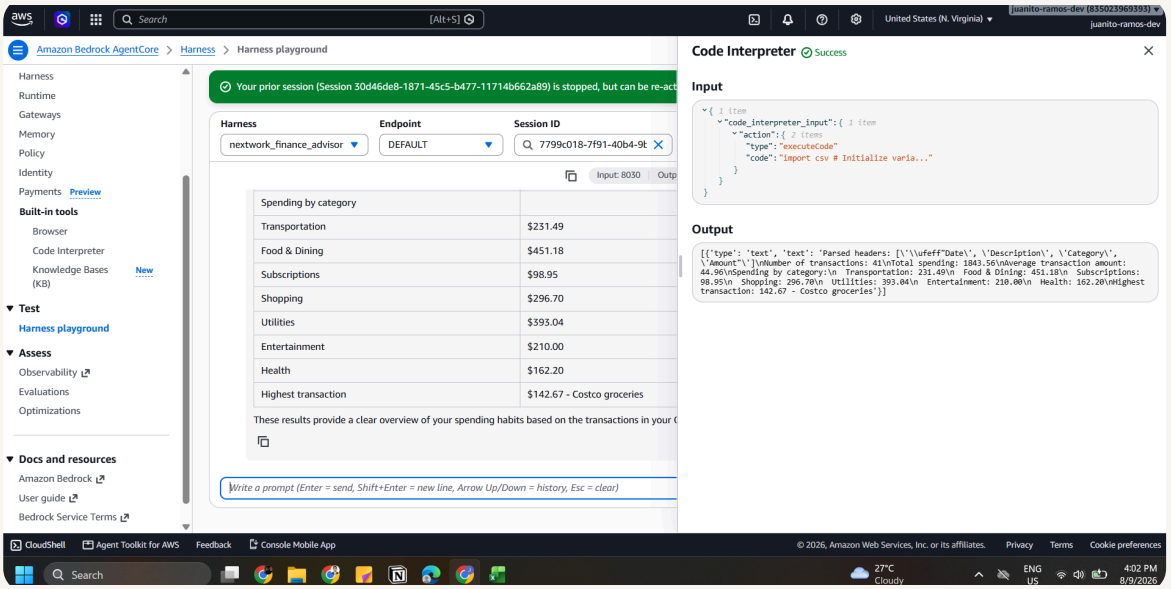
JUANITO RAMOS II
NextWork Student

Clear and specific instructions make the agent's responses more consistent, useful, and relevant. In this project, adding formatting and percentage requirements improved the spending analysis and made it easier to understand and compare categories.



Analyzing Spending Data with Code Interpreter

In this step, we put the AI finance agent to work on real transaction data. We uploaded the transactions CSV to the Code Interpreter, asked the agent to calculate spending by category and recommend budgets, and examined the agent trace to see how it used Python to analyze the data. We then iterated on the agent's instructions to improve its responses by requiring consistent dollar formatting and percentage breakdowns.



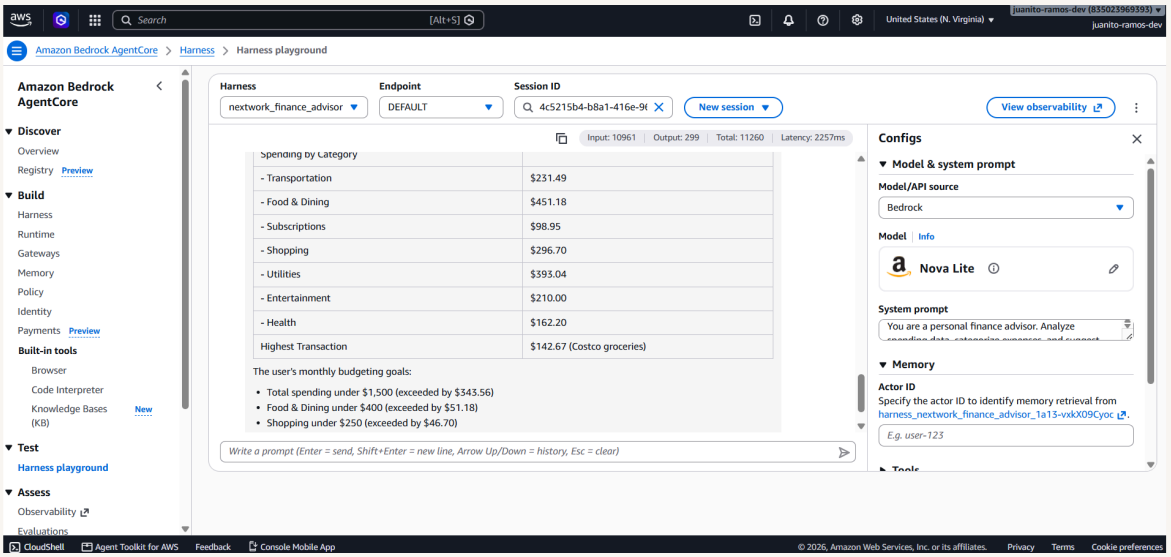
How Code Interpreter processed the data

Code Interpreter helped the agent analyze my spending data by allowing it to write and execute Python code to process the transactions CSV. It calculated the total spending, average transaction amount, spending by category, and highest transaction, then used those results to identify areas where I was overspending and provide budget recommendations. This allowed the agent to perform data analysis automatically without me having to write the code myself.

JUANITO RAMOS II
NextWork Student

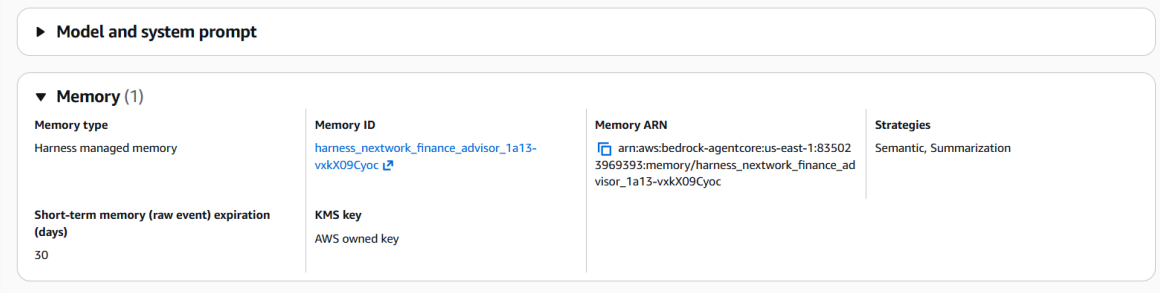
Iterating on Agent Instructions with Traces

I changed the agent's instructions by adding formatting requirements: dollar amounts should always be displayed with two decimal places, and each spending category should include a percentage showing its share of total spending. This makes the financial analysis more consistent and easier to understand because I can see both the amount spent and the relative impact of each category.



Enabling Cross-Session Agent Memory

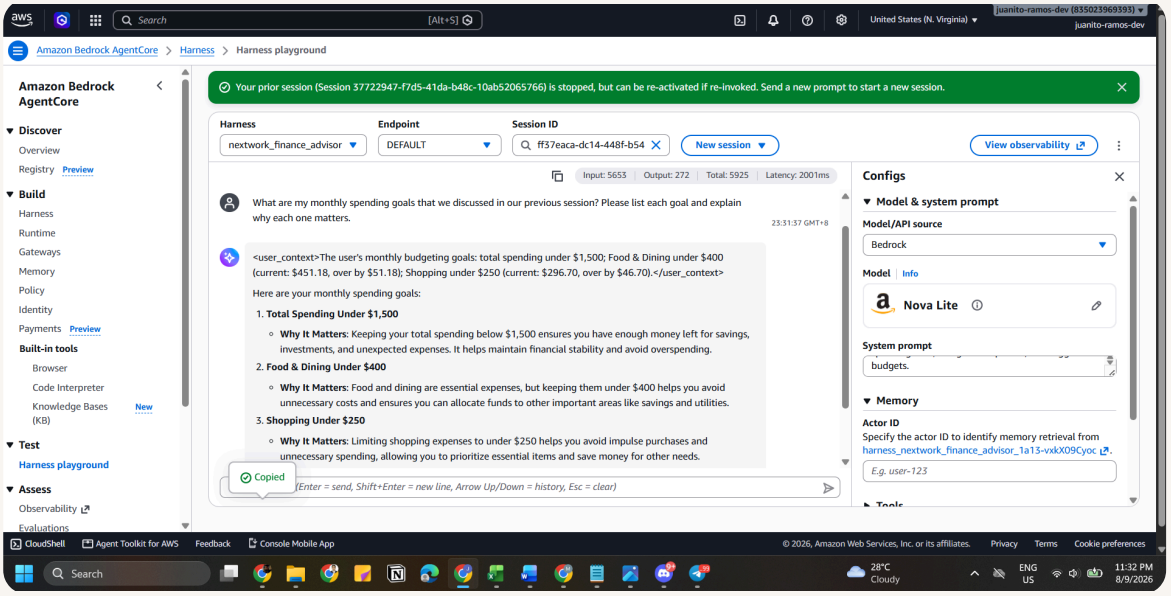
We are enabling Agent Memory with session summarization so the finance agent can retain important information from previous conversations. The goal is to make the agent remember a user's spending preferences, budget limits, and financial goals across separate sessions. Cross-session memory is important because users should not have to repeat their financial goals every time they start a new conversation. For example, the agent can remember that the user wants to keep total spending below $1,500, Food & Dining below $400, and Shopping below $250, then use those goals when giving recommendations in a new session.



Session summarization vs full history

Session summarization condenses the important information from a previous conversation into a summary that can be used in future sessions. This allows the finance agent to remember relevant spending patterns, budget goals, and preferences without needing the user to repeat them. It is useful because a

This makes the agent more personalized and useful over time.



Testing cross-session recall

The agent recalled information from the previous session through AgentCore Memory with session summarization. It remembered my monthly budgeting goals, including keeping total spending under $1,500, Food & Dining under $400, and Shopping under $250.

It also remembered my previous spending of $451.18 on Food & Dining and $296.70 on Shopping, allowing it to recognize that I was over budget in both categories.

Wrapping Up

I did this project today to learn how to build and configure an AI agent using Amazon Bedrock, give it tools such as Code Interpreter, and use memory to maintain context across sessions. Another skill I want to learn is how to build more advanced AI agents that can connect to external services and databases, use multiple tools, and automate real-world workflows.

The place to learn & showcase your skills

Check out nextwork.ai for more projects

